Analysis and Design of Algorithms

CS3230

Recitation

Week 4

Prof Lee Wee Sun

Design Paradigm

Divide, **conquer**, **combine**.

Consider Master Theorem recurrence

$$T(n) = \boldsymbol{a}T(\boldsymbol{n}/\boldsymbol{b}) + \boldsymbol{f}(\boldsymbol{n})$$

Reduce #sub-problems

Reduce sub-problem size

Reduce time to divide and combine

Examples

Binary search: $T(n) = \Theta(\lg n)$

$T(n) = 2T(n/2) + \Theta(1)$ ⇨ $T(n) = T(n/2) + \Theta(1)$

Powering, Fibonacci Num: $T(n) = \Theta(\lg n)$

$T(n) = 2T(n/2) + \Theta(1)$ ⇨ $T(n) = T(n/2) + \Theta(1)$

Matrix Mult: $T(n) = \Theta(n^{\log(2)\,7})$

$T(n) = 8T(n/2) + \Theta(n^2)$ ⇨ $T(n) = 7T(n/2) + \Theta(n^2)$

VLSI Layout: $T(n) = \Theta(\sqrt{n})$

$W(n) = 2W(n/2) + \Theta(1)$ ⇨ $W(n) = 2(n/4) + \Theta(1)$

Admin

- Programming assignment 1 released. Deadline Friday 16 September

MCQ

Question 1

Question 9912

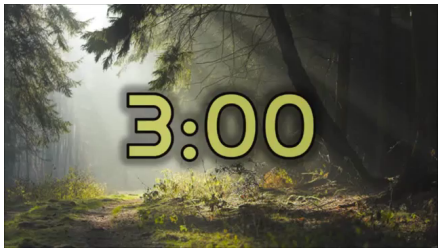
A divide and conquer algorithm for solving problem has recurrence $T(n) = 8T(n/2) + n^2$. Two improvements to the algorithm are found.

Improvement I: The time for divide and combine is reduced to $n^2/2$.

Improvement II: The runtime for divide and combine remains the same, but the number of subproblems a is reduced to 4.

Which of the improvements is asymptotically better?

- Improvement I
- Improvement II
- The two improvements result in algorithms with the same asymptotic runtimes.



Solution

$$T(n) = 8T(n/2) + n^2$$

$$f(n) = n^2 = O(n^{\log_2 8 - \epsilon})$$

Case 1:

$$T(n) = \Theta(n^3)$$

Original. Improvement I does not change asymptotic runtime

$$T(n) = 4T(n/2) + n^2$$

$$f(n) = n^2 = \Theta(n^{\log_2 4})$$

Case 2:

$$T(n) = \Theta(n^2 \lg n)$$

Improvement II gives substantial improvement

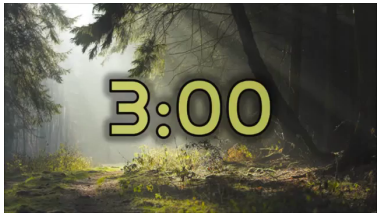
Question 2

```
Power1(a,k)
if k == 1
    return a
k′=⌊k/2⌋
if k is odd
    return a*Power1(a,k′)*Power1(a,k’)
else
    return Power1(a,k′)*Power1(a,k′)
```

```
Power2(a,k)
if k == 1
    return a
k′=⌊k/2⌋
p = Power2(a,k’)
if k is odd
    return a*p*p
else
    return p*p
```

Which of the methods is asymptotically faster?

- Power1
- Power2
- The runtimes are asymptotically the same.



Solution

$$T(n) = 2T(n/2) + \Theta(1)$$

$$f(n) = 1 = O(n^{\log_2 2 - \epsilon})$$

Case 1:

$$T(n) = \Theta(n)$$

Power1: Solve 2 subproblems

$$T(n) = T(n/2) + \Theta(1)$$

$$f(n) = 1 = \Theta(n^{\log_2 1})$$

Case 2:

$$T(n) = \Theta(\lg n)$$

Power2: Reduce the number of subproblems to 1.

Question 3

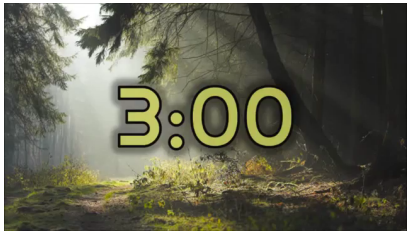
Consider the following method of using a heap for merging two sorted lists in merge sort:

Method I: Construct a min heap using both lists. Extract the minimum elements until the heap is empty.

Method II: Put the first element in each list in a min heap. Extract the minimum element and replace the element by inserting the next element from the same list. Repeat until all elements have been inserted and extracted from the heap.

Which of the methods give a better asymptotic running time when used in merge sort?

- Method I
- Method II
- The two methods give the same asymptotic runtime.



Solution

$$T(n) = 2T(n/2) + \Theta(n \lg n)$$

$$f(n) = n \lg n = \Theta(n^{\log_2 2} \lg n)$$

Case 2:

$$T(n) = \Theta(n \lg^2 n)$$

Method 1: Build heap then extract all elements take $\Theta(n \lg n)$ time.

$$T(n) = 2T(n/2) + \Theta(n)$$

$$f(n) = n \lg n = \Theta(n^{\log_2 2})$$

Case 2:

$$T(n) = \Theta(n \lg n)$$

Method 2: Heap size is always at most 2. So inserting and extracting all elements take $\Theta(n)$ time.

Question 4

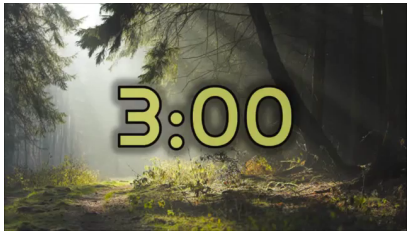
Ali has n coconuts. All of them are the same weight, except one which is heavier. He has a super balance scale that can fit an *arbitrary* number of coconuts on both sides. Consider the following ways of finding the heavy coconut:

Method I: Divide the coconuts into two equal sized piles and put each pile on one side of the scale. Keep the heavier pile and repeat until the heavy coconut is found.

Method II: Divide the coconuts into three equal sized piles and put two of the piles on the two sides of the scale. If they are equally heavy, repeat with the third pile, otherwise repeat with the heavier pile, until the heavy coconut is found.

Which method requires fewer number of steps to find the heavy coconut?

- Method I
- Method II
- Both methods require the same number of steps.



Solution

Method 1: $T(n) = T(n/2) + 1$

One subproblem, height of recursion tree is $\lceil \log_2 n \rceil$
One operation in each node, so total number of operations is $T(n) = \lceil \log_2 n \rceil$

Smaller subproblem!

Method 2: $T(n) = T(n/3) + 1$

One subproblem, height of recursion tree is $\lceil \log_3 n \rceil$
One operation in each node, so total number of operations is $T(n) = \lceil \log_3 n \rceil$

Question 5

Who is the Master of Algorithms pictured below?

- Charles Leiserson
- Ronald Rivest
- Erik Demaine
- Edsger Dijkstra

Question 5

Erik Demaine

Professor at MIT.
Former child prodigy.
Joined MIT in 2001 at age 20 as the youngest professor in the history of MIT.
Works on computer origami, data structures, computational geometry, ..., awarded MacAuthur Fellowship (“genius award”).

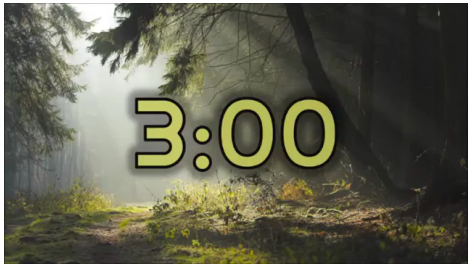
Structured Questions

Question 1(a)

The Hamadard matrices are defined as follows:

- H_0 is a 1×1 matrix $[1]$.
- For $k > 0$, H_k is a $2^k \times 2^k$ matrix

$$H_k = \begin{bmatrix} H_{k-1} & H_{k-1} \\ H_{k-1} & -H_{k-1} \end{bmatrix}$$




```
MULT(k, H_k, x)
if k == 0
    return x
x_1 = x[1..2^{k-1}]
x_2 = x[2^{k-1} + 1..2^k]
y_1 = MULT(k - 1, H_{k-1}, x_1) + MULT(k - 1, H_{k-1}, x_2)
y_2 = MULT(k - 1, H_{k-1}, x_1) - MULT(k - 1, H_{k-1}, x_2)
y[1..2^{k-1}] = y_1
y[2^{k-1} + 1..2^k] = y_2
return y
```

What is the runtime of MULT?

Solution

Let $n = 2^k$ be the length of the vector. We solve 4 subproblems of size $n/2$. The time to divide and combine includes time to add the vectors of size $n/2$ together. Hence

$T(n) = 4T(n/2) + \Theta(n)$

To apply Master theorem, check:

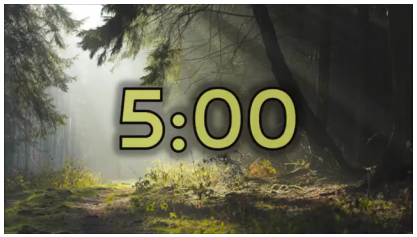
$$f(n) = n = O(n^{\log_2 4 - \epsilon})$$

Case 1 applies, giving:

$T(n) = \Theta(n^2)$

Question 1(b)

Design a faster algorithm for solving the problem.



The Hamadard matrices are defined as follows:

- H_0 is a 1×1 matrix $[1]$.
- For $k > 0$, H_k is a $2^k \times 2^k$ matrix

$$H_k = \begin{bmatrix} H_{k-1} & H_{k-1} \\ H_{k-1} & -H_{k-1} \end{bmatrix}$$

```
MULT(k, H_k, x)
if k == 0
    return x
x_1 = x[1..2^{k-1}]
x_2 = x[2^{k-1} + 1..2^k]
y_1 = MULT(k - 1, H_{k-1}, x_1) + MULT(k - 1, H_{k-1}, x_2)
y_2 = MULT(k - 1, H_{k-1}, x_1) - MULT(k - 1, H_{k-1}, x_2)
y[1..2^{k-1}] = y_1
y[2^{k-1} + 1..2^k] = y_2
return y
```

Previous

Solution

```
MULT(k, H_k, x)
    if k == 0
        return x
    x_1 = x[1..2^{k-1}]
    x_2 = x[2^{k-1} + 1..2^k]
    z_1 = MULT(k - 1, H_{k-1}, x_1)
    z_2 = MULT(k - 1, H_{k-1}, x_2)
    y[1..2^{k-1}] = z_1 + z_2
    y[2^{k-1} + 1..2^k] = z_2 - z_2
    return y
```

$T(n) = 2T(n/2) + \Theta(n)$

To apply Master theorem, check:

$$f(n) = n = \Theta(n^{\log_2 2})$$

Case 2 applies, giving:

$T(n) = \Theta(n \lg n)$

Question 2(a)

Given a **sorted** array $A[1..n]$ of distinct integers, is there an index i where $A[i] = i$?

```
FIND(low, high, A)
if high < low
    return false
mid = ⌊(low + high)/2⌋
if A[mid] == mid
    return true
return FIND(low, mid − 1, A) or FIND(mid + 1, high, A)
```

What is the asymptotic runtime of FIND?

Solution

$T(n) = 2T(n/2) + \Theta(1)$

To apply Master theorem, check:

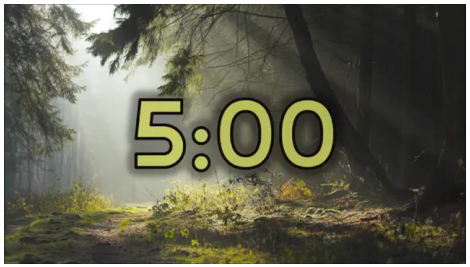
$$f(n) = 1 = O(n^{\log_2 2 - \epsilon})$$

Case 1 applies, giving:

$T(n) = \Theta(n)$

Question 2(b)

Reminder: Sorted array of distinct integers.
Design a faster algorithm for solving the problem.




```
FIND(low, high, A)
if high < low
    return false
mid = ⌊(low + high)/2⌋
if A[mid] == mid
    return true
return FIND(low, mid − 1, A) or FIND(mid + 1, high, A)
```

Previous

Solution

Consider the transformed array $A[i]$-i.

If there is an index such that $A[i] = i$ then the value of that element is 0.

We argue that the array $A[i]$-i is sorted.

Consider the difference between consecutive elements in the transformed array:

$A[i+1]-(i+1)-(A[i]-i) = A[i+1]-A[i]-1$.

The elements of A are distinct

integers and A is sorted: difference ≥ 0.

This implies that the transformed array is non-decreasing, and hence sorted.

We can perform binary search on the transformed array to see if 0 exists (reduce the num of subproblems to one).

Doing the transformation on the whole array will take $\Theta(n)$ time.

Instead, we compute the value of $A[i]$-i within the binary search code.

```
FIND(low, high, A)
    if low < high
        return false
    mid = ⌊(low + high)/2⌋
    if A[mid] == mid
        return true
    if A[mid] − mid > 0
        return FIND(low, mid − 1, A)
    return FIND(mid + 1, high, A)
```

$T(n) = T(n/2) + \Theta(1)$

To apply Master theorem, check:

$$f(n) = 1 = \Theta(n^{\log_2 1})$$

Case 2 applies, giving:

$T(n) = \Theta(\lg n)$